

The Alpha Engine

POC Specification — v1.0

Internal market-making simulation via agent-based model and Dutch auction clearing

This document is a complete implementation specification for the Alpha Engine POC. It is written for a developer or LLM tasked with building the simulation from scratch. Every design decision is stated explicitly. Where a choice remains open it is flagged. Nothing needs to be inferred.

PART I — WHAT WE ARE BUILDING

1. The Core Idea

A closed market simulation where two populations of agents — longs and shorts — continuously try to exchange EUR and BTC. No price is programmed. A price emerges from the agents' collective activity through a Dutch auction clearing mechanism. The goal is to observe the emergent internal price p_{int} and the dynamics it produces: stability, oscillation, liquidity crises, agent mortality.

The system is a complexity-science demonstration: simple local rules at the agent level, rich collective behaviour at the market level.

2. Scope of This POC

This POC implements exactly the following — nothing more:

- An internal agent pool only. No external price feed. No external traders.
- Two fixed agent populations: longs (hold EUR, want BTC) and shorts (hold BTC, want EUR).
- A carry-cost pressure mechanism that drives agents to trade.
- A Dutch auction that clears the internal order queue each cycle and produces p_{int} .
- Agent mortality: agents who exhaust their capital exit. No replenishment.
- Observable outputs: p_{int} time series, queue depth, agent population, aggregate capital, P&L; distribution.

Explicitly deferred to future versions:

- External price feed p_{ext} and external traders.
- Spread mechanics and the bid/ask quote.
- Bilateral yield curves (two curves, one per currency).
- Capital replenishment / wealth recycling.
- Risk warehousing across multiple time horizons.

3. Agent Structure

There are $2n$ agents in total: n long agents and n short agents. Each agent is a simple data structure with the following fields:

Field	Type	Description
id	int	Unique identifier
side	enum	LONG or SHORT — fixed for the lifetime of the agent
eur_i	float	EUR balance
btc_i	float	BTC balance
K_i	float	Total capital in EUR terms: $\text{eur}_i + \text{btc}_i * x_0$ (updated each tick using p_{int})
K0_i	float	Initial total capital in EUR terms — retained for threshold scaling
phi_i	float	Accumulated carry pressure — the agent's internal clock
d_i	float	Firing threshold — scales with $K0_i$
alive	bool	False when K_i falls below bankruptcy threshold

*Agents never switch sides. A long remains long until bankruptcy. K_i is a derived quantity — always $\text{eur}_i + \text{btc}_i * p_{\text{int}}(t)$ — used only for threshold scaling and bankruptcy checks. ϕ_i uses the Greek letter phi (pressure) deliberately to avoid confusion with the price variables p_{int} (lowercase p).*

4. Initial Capital Distribution

Agent capital is drawn from a power-law (Pareto) distribution. This reflects the empirical size distribution of real market participants — many small agents, few large ones.

```
K0_i ~ Pareto(x_min, alpha) x_min = minimum capital in EUR terms (baseline size)
alpha = scaling exponent (suggested starting value: 1.5) Draw n values for longs,
n values for shorts. Rescale so each side sums to K/2 in EUR terms. Initialise
balances using starting price x_0: LONG agents (EUR-heavy, want to sell EUR):
eur_i = K0_i * f_eur # e.g. f_eur = 0.9 btc_i = K0_i * (1 - f_eur) / x_0 SHORT
agents (BTC-heavy, want to sell BTC): btc_i = K0_i * f_btc / x_0 # e.g. f_btc =
0.9 eur_i = K0_i * (1 - f_btc) f_eur, f_btc = home currency fraction (global
parameter, suggested 0.9) x_0 = initial EUR/BTC price (global parameter)
```

x_0 sets the exchange rate for converting between EUR and BTC at initialisation. $K0_i$ is always expressed in EUR terms. The home currency fraction f determines how EUR-heavy or BTC-heavy agents start — 1.0 means pure home currency, 0.5 means evenly split.

5. Agent Roles — Lane Model

Agents stay in their designated role for the entire simulation. This is the Lane model — no inventory accumulation, no bias flipping.

Side	Starts	Mandate	Fires a...	Accumulates
LONG	EUR-heavy	Sell EUR, buy BTC	Sell order (EUR)	BTC over time
SHORT	BTC-heavy	Sell BTC, buy EUR	Buy order (BTC)	EUR over time

Both sides hold both currencies throughout the simulation. A long starts EUR-heavy and accumulates BTC as it trades — but it never flips its mandate. It continues to offer EUR for BTC regardless of how much BTC it has accumulated. Carry cost is on the home currency (EUR for longs, BTC for shorts), reflecting the cost of holding the position they are trying to unwind.

PART III — THE PRESSURE MECHANISM

6. Carry Cost and Pressure Accumulation

Each agent has a carry cost c — a fixed rate paid per tick for holding their position. For this POC, c is uniform across all agents. Carry cost is not deducted from capital directly each tick; instead it accumulates as pressure ϕ_i . Trading relieves the pressure.

Each tick, for every alive agent i : $\phi_i(t) = \phi_i(t-1) + c$ c is a global parameter — same for all agents.

c is intentionally decoupled from K_i here. The firing frequency difference between small and large agents comes entirely from the threshold d_i , not from c . This keeps the two mechanisms cleanly separable.

7. Firing Threshold — Scales With Capital

The threshold d_i scales with the agent's initial capital $K0_i$. Large agents have a higher threshold — they accumulate more pressure before firing. Small agents hit their threshold faster and trade more frequently. This directly mirrors the capital scaling law: small agents are more active.

$d_i = d_{base} * (K0_i / K0_{min})$ d_{base} = threshold for the smallest agent (global parameter) $K0_{min}$ = capital of the smallest agent in the population Result: firing frequency of agent i is inversely proportional to $K0_i$.

The smallest agent fires every d_{base} / c ticks. An agent with twice the minimum capital fires half as often. The largest agent fires most rarely.

8. Firing and Reset

When $\phi_i(t) \geq d_i$, the agent fires a trade order and resets its pressure:

if $\phi_i(t) \geq d_i$: place_order(agent_i) $\phi_i(t) = 0$

Full reset to zero. Partial reset (carrying forward residual pressure) is a future variant but not implemented here.

PART IV — TRADE SIZE

9. Order Size Rule

When an agent fires, the size of its order depends on whether its current capital is above or below the baseline capital $K0_min$ (the smallest agent's initial capital). This creates a natural two-regime system:

```
if K_i >= K0_min: # above baseline order_size = K_i / q # fractional trade else: #  
below baseline – go all-in order_size = K_i # full remaining capital q = 16  
(global parameter – fraction denominator)
```

Regime	Condition	Order size	Behaviour
Above baseline	$K_i \geq K0_min$	K_i / q	Agent trades a fixed fraction each time; capital depletes slowly
Below baseline	$K_i < K0_min$	K_i (all-in)	Agent bets everything; one trade determines survival

The below-baseline regime creates a natural mortality mechanism. Small or depleted agents make one final all-in trade. If the trade goes against them, they go bankrupt immediately.

PART V — THE CLEARING MECHANISM

10. The Order Queue

When an agent fires, its order is placed in a shared queue. Orders are not matched immediately — they accumulate over a rolling window of W ticks before the auction runs. This ensures the auction has sufficient order flow to find a genuine crossing price even during sparse periods.

```
queue = list of (agent_id, side, size, tick_placed) Orders older than W ticks are  
expired and removed. W = clearing window (global parameter, suggested: 5 ticks)
```

11. Dutch Auction Clearing

At the end of every tick, the house runs a Dutch auction over the current queue. The auction discovers the price at which cumulative long demand meets cumulative short supply.

Auction procedure:

- Separate the queue into long orders (offering EUR) and short orders (offering BTC).
- Sort both sides by size descending — largest orders are matched first.
- Sweep price upward from $p_int(t-1)$. At each candidate price, compute cumulative volume on each side.
- The clearing price $p_int(t)$ is the lowest price at which cumulative short supply $\geq$ cumulative long demand.
- All orders up to the clearing volume transact at $p_int(t)$. Residual unmatched orders remain in the queue.
- If no crossing is found (one side is empty or no overlap), $p_int(t) = p_int(t-1)$. Queue carries forward.

The auction clears by volume — order sizes in capital units, not number of orders. A single large agent firing can match many small agents on the opposite side.

12. Price Formation

```
p_int(t) = Dutch auction clearing price this tick
If auction finds no crossing:
p_int(t) = p_int(t-1) # price holds
p_int(0) = p_0 # initial price, global parameter
```

p_{int} is entirely endogenous. It is not set, targeted, or adjusted by any external mechanism. Persistent imbalance between long and short flow will cause p_{int} to trend. Symmetric flow produces a stable or noisy price. The shape of p_{int} is what we are studying.

PART VI — CAPITAL DYNAMICS & MORTALITY

13. Capital Update After a Trade

When two orders match at $p_{int}(t)$, capital changes hands. For a long agent selling EUR_amount of EUR and a short agent selling BTC_amount of BTC:

```
When a long order (size = EUR_amount) matches a short order:
BTC_amount = EUR_amount / p_int(t)
long agent i: eur_i -= EUR_amount
btc_i += BTC_amount
short agent j: btc_j -= BTC_amount
eur_j += EUR_amount
K_i = eur_i + btc_i * p_int(t) # recompute total capital in EUR terms
K_j = eur_j + btc_j * p_int(t)
```

Both agents now hold both currencies. The exchange is at the current p_{int} . No value is created or destroyed in the trade — it is zero-sum between the two agents. The only non-zero-sum flow is carry cost, which drains the pool to the house.

P&L; is not tracked. Within this closed internal market all trading P&L; is zero-sum and the aggregate is always zero. The informative measure of pool health is $total_capital(t)$ — the aggregate drain from carry cost.

14. Carry Cost Deduction

Carry cost is deducted from capital each tick, separately from trading:

```
Each tick, for every alive agent i:
LONG: eur_i -= c * eur_i # carry on EUR
holding SHORT: btc_i -= c * btc_i # carry on BTC
K_i = eur_i + btc_i * p_int(t) # recompute total capital
c is a global parameter — same for all agents.
```

Open: proportional carry ($c * home_balance$) vs flat (c_abs per tick). Proportional is more realistic; flat is simpler.

15. Bankruptcy and Agent Death

```
bankruptcy threshold = epsilon (global parameter, e.g. 0.01 * x_min)
K_i = eur_i + btc_i * p_int(t) # total capital in EUR terms
if K_i <= epsilon: agent_i.alive = False
remove all agent_i orders from queue
```

Dead agents are removed from the simulation permanently. No replenishment in this version. The pool shrinks monotonically. Simulation ends when all agents are dead or T ticks have elapsed.

PART VII — SIMULATION LOOP

16. The Main Loop

```
Initialise: Draw  $K0_i$  from  $\text{Pareto}(x_{\min}, \alpha)$ , rescale to sum  $K/2$  per side Set
eur_i, btc_i using  $x_0$  and home fraction  $f$  Set  $d_i = d_{\text{base}} * (K0_i / K0_{\min})$  for
each agent Set  $\phi_i = 0$  for all agents Set  $p_{\text{int}}(0) = x_0$  queue = [] For  $t = 1$  to
 $T$ : 1. CARRY ACCUMULATION For each alive agent  $i$ :  $\phi_i += c$  if LONG:  $\text{eur}_i -= c * \text{eur}_i$  if SHORT:  $\text{btc}_i -= c * \text{btc}_i$   $K_i = \text{eur}_i + \text{btc}_i * p_{\text{int}}(t-1)$  2. FIRING
CHECK For each alive agent  $i$ : if  $\phi_i \geq d_i$ : home = eur_i if LONG else btc_i
order_size = home/ $q$  if  $K_i \geq x_{\min}$  else home add (agent_i, side_i, order_size,  $t$ )
to queue  $\phi_i = 0$  3. QUEUE EXPIRY Remove orders with  $\text{tick\_placed} < t - W$  4. DUTCH
AUCTION  $p_{\text{int}}(t) = \text{run\_auction}(\text{queue})$  Update eur_i, btc_i for matched agents
Recompute  $K_i$  for matched agents Remove matched orders from queue 5. BANKRUPTCY
CHECK For each alive agent  $i$ :  $K_i = \text{eur}_i + \text{btc}_i * p_{\text{int}}(t)$  if  $K_i \leq \epsilon$ :
mark dead, purge orders 6. RECORD Log  $p_{\text{int}}(t)$ , queue depth, alive count, total
capital 7. TERMINATION CHECK if no alive agents: break
```

PART VIII — PARAMETERS

17. Complete Parameter Table

Parameter	Suggested value	Description
n	50	Number of agents per side (100 total)
K	1,000,000	Total initial capital in EUR terms (split $K/2$ per side)
x_0	1.0	Initial EUR/BTC price — used for initialisation and as $p_{\text{int}}(0)$
α	1.5	Pareto exponent for capital distribution
$x_{\min}$	$K / (n * 10)$	Pareto minimum capital in EUR terms (baseline size)
f	0.9	Home currency fraction at initialisation
c	0.001	Carry cost rate per tick on home currency balance
d_{base}	1.0	Firing threshold for the smallest agent
q	16	Order size denominator (above-baseline agents trade home/q)
W	5	Clearing window in ticks
ϵ	$0.01 * x_{\min}$	Bankruptcy threshold in EUR terms
T	10,000	Maximum simulation ticks

PART IX — OUTPUTS & SUCCESS CRITERIA

18. Required Outputs

The simulation must record the following time series at every tick:

Output	Description
p_int(t)	Internal clearing price from the Dutch auction
queue_depth(t)	Number of unmatched orders in the queue (long and short separately)
alive_count(t)	Number of surviving agents (long and short separately)
total_capital(t)	Sum of K_i across all alive agents in EUR terms — the pool drain curve
matched_volume(t)	Total volume cleared in the auction this tick

Minimum viable visualisation (matplotlib):

- p_int(t) as a time series — the primary output.
- alive_count(t) for longs and shorts — shows mortality dynamics.
- total_capital(t) — shows the pool drain rate.
- queue_depth(t) long vs short — reveals imbalance and liquidity crises.

19. What We Are Looking For

Success is not a correct price. It is interesting, legible emergent behaviour:

- Does p_int emerge as a stable, noisy, or oscillatory process?
- How long does the pool survive before agent mortality collapses the dynamics?
- Do liquidity crises appear — queue builds on one side with no counterparty, sudden price gaps?
- Do small agents die first (carry drain) or large agents (slow firing, missing good prices)?
- Does the system show any self-organising behaviour before collapse — clustering, oscillation, calm periods?
- Does the Dutch auction behave differently from naive VWAP — does it produce sharper price moves?

PART X — OPEN DECISIONS (not blocking v1)

20. Deferred Choices

The following decisions are not needed for v1 but should be revisited once baseline dynamics are understood:

Decision	Options	Why deferred
----------	---------	--------------

Carry deduction	Proportional ($c * K_i$) vs flat (c_{abs})	Both work; observe mortality dynamics first
Pressure reset	Full ($\phi=0$) vs partial ($\phi=\phi-d_i$)	Changes firing rhythm subtly; isolate later
Per-agent threshold	Linear scaling vs power-law d_i	Linear scaling sufficient for v1
Bilateral yield curves	One curve vs EUR-earn / BTC-pay	Adds asymmetry; study symmetric case first
Spread mechanics	No spread vs bid/ask around p_{int}	Only needed when external traders are added
External traders	None vs one vs competing arbitrageurs	V2 — couples p_{int} to p_{ext}
Capital replenishment	None vs yield payments vs injection	V2 — extends pool lifetime
Soft reset / Option B	Fixed trade size vs capital consumption	Richer dynamics; add after v1 baseline

PART XI — CODE ARCHITECTURE

21. Module Structure

The codebase is split into five modules plus a thin entry point. Each module has a single responsibility. No module imports from another except through the interfaces described below.

Module	Class(es)	Responsibility
config.py	Config	All parameters and switches in one place. Single source of truth.
agents.py	Agent, Population	Agent state and behaviour. No market logic.
market.py	Queue, Auction	Order queue, expiry, Dutch auction, price formation.
simulation.py	Simulation	Main loop. Wires modules together. No business logic.
analysis.py	Recorder, Analyser	Time series recording, plots, sanity checks.
main.py	—	Entry point. Three lines: load config, run, show results.

22. Config Design

All parameters and all open-decision switches live in config.py as a single dataclass. A developer changes one file to reconfigure the entire simulation. Derived parameters (x_{min} , ϵ) are computed automatically if not set explicitly.

```
@dataclass class Config: # Population n: int = 50 K: float = 1_000_000 x_0: float = 1.0 f: float = 0.9 # Capital distribution alpha: float = 1.5 x_min: float = None
```



```
# computed: K / (n * 10) if None # Pressure c: float = 0.001 d_base: float = 1.0 #
Trade q: int = 16 # Clearing W: int = 5 beta: float = 0.01 # price impact of
imbalance # Mortality epsilon: float = None # computed: 0.01 * x_min if None #
Simulation T: int = 10_000 seed: int = 42 # None = fresh random run # Switches
(open decisions) carry_proportional: bool = True # True = c*home_balance, False =
flat c
```

23. Reproducibility — Seed

The simulation has two sources of non-determinism: the Pareto capital draw at initialisation, and tie-breaking when agents have identical order sizes. Both are controlled by a single global seed.

```
# In simulation.py, before anything else: if config.seed is not None:
numpy.random.seed(config.seed) # Tie-breaking in the auction sort:
orders.sort(key=lambda o: (o.size, o.agent_id), reverse=True) # agent_id is unique
and stable – sort is always well-defined
```

seed=42 gives a fully reproducible run. seed=None gives a fresh random population each time. Comparing runs across seeds tests whether dynamics are structural or seed-dependent.

24. Sanity Checks

analysis.py runs automated assertions after every simulation. These fail loudly if something is wrong and are the primary correctness check for the POC.

- total_capital(t) is monotonically non-increasing — carry drain only ever removes capital.
- p_int(t) is always positive.
- matched_volume(t) <= queue_depth(t) at every tick.
- alive_count_long(t) + alive_count_short(t) <= 2n at every tick.
- Per-trade capital conservation: sum of K_i before trade == sum of K_i after trade (zero-sum check).
- No agent with alive=False appears in the queue.

25. Price Feed Abstraction (V2)

In v1 there is no external price. In v2, external traders need to observe p_ext each tick. Rather than hardcoding a Gaussian random walk into the simulation loop, the feed is an abstraction in feeds.py — a class with a single method next() that returns the next price. This makes the feed swappable without touching the loop.

```
# feeds.py class GaussianFeed: def __init__(self, config): self.p = config.x_0
self.mu = config.feed_mu # drift self.sigma = config.feed_sigma # volatility def
next(self): self.p *= exp(normal(self.mu, self.sigma)) return self.p # Swappable
alternatives (same interface): # ReplayFeed – replays historical price data #
JumpFeed – Gaussian with occasional fat-tail jumps # LiveFeed – connects to a real
market data source
```

The simulation loop in v2 calls feed.next() once per tick and passes the result only to the external trader agents — internal agents never see it. The internal pool remains fully decoupled from p_ext by design.

The tick itself remains a bare range(T) loop in v1 and v2. Intrinsic time is enforced implicitly by the threshold mechanism — each agent fires on its own schedule determined by d_i and c . A true event-driven clock is only needed if the simulation is later extended to react to external events in real time.